

AI-Driven Real-Time Interview Simulation and Recruitment System

A project report submitted in partial fulfillment of the requirements
for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

By

NAGA SUBRAHMANYAM

23VV1F0016

Under the Esteemed Guidance of

Project Guide

DEPARTMENT OF INFORMATION TECHNOLOGY

JNTUGV College of Engineering Vizianagaram (Autonomous)

Jawaharlal Nehru Technological University, Gurajada Vizianagaram

Dwarapudi, Vizianagaram - 535003, Andhra Pradesh, India

2025–2026

CERTIFICATE

This is to certify that the project report entitled “AI-Driven Real-Time Interview Simulation and Recruitment System” submitted by NAGA SUBRAHMANYAM bearing registration number 23VV1F0016 is prepared in partial fulfillment for the award of the degree of Master of Computer Applications. The work presented in this report was carried out as a project study on a full-stack interview simulation and recruitment platform using modern web technologies, artificial intelligence services, and real-time communication components.

The contents of this report describe the design, implementation, testing, and analysis of the developed system. The results embodied in this project have not been submitted to any other university or institute for the award of any degree or diploma.

Signature of the Supervisor

Signature of the Head of the Department

Project Guide

Head of the Department

External Signature

DECLARATION

I, NAGA SUBRAHMANYAM with Reg. No: 23VV1F0016, declare that this submission represents my own project work and report writing. Where external technologies, documentation, or standard references have been used, they are acknowledged in the references section. I also declare that I have not misrepresented or falsified any project implementation, data, or result described in this report.

Date:

Place:

(Signature)

NAGA SUBRAHMANYAM

23VV1F0016

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide and faculty members for their guidance, encouragement, and support throughout the completion of this project. Their suggestions helped in refining the problem statement, improving the design, and presenting the implementation in an organized academic form.

I also thank the Department of Information Technology and the institution for providing the academic environment required to complete this work. The project required knowledge of web development, databases, artificial intelligence, real-time communication, and software testing, and the learning support provided by the department was valuable.

Finally, I thank my family, friends, and peers for their motivation and feedback during the development and documentation of the AI-driven interview simulation system.

ABSTRACT

Interview preparation and recruitment evaluation are increasingly dependent on digital platforms, yet many existing systems handle only isolated tasks such as video calling, coding practice, or resume screening. This project presents an AI-driven real-time interview simulation and recruitment system that combines mock interviews, coding interviews, peer practice, recruiter-led sessions, AI coaching, learning roadmaps, resume handling, and performance reporting inside one browser-based application.

The system is developed using React, Vite, and Tailwind CSS for the frontend, Node.js and Express for backend APIs, MongoDB and Mongoose for persistence, Socket.IO and WebRTC for real-time collaboration, Web Speech API for speech input, browser camera analysis for monitoring, and Gemini AI services for question generation, coaching, and evaluation support. The platform supports authentication with email verification, live AI interviews, LeetCode-style coding practice, competitive coding battle rooms, peer mock interview requests, recruiter interview rooms, AI-generated reports, and personalized learning roadmaps.

The project aims to reduce fragmented interview preparation by offering a unified workflow that records answers, evaluates responses, tracks progress, and provides actionable feedback. Results show that the application can provide a structured interview experience with automated question flow, real-time user interaction, and stored performance history. The system is suitable for students, job seekers, training institutions, and recruiters who need a scalable interview preparation and assessment environment.

Table of Contents

CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
1 INTRODUCTION	1
1.1 Introduction to Interview Simulation	1
1.2 Artificial Intelligence in Recruitment	2
1.3 Problem Statement	3
1.4 Objectives of the Project	5
1.5 Scope of the Project	6
1.6 Organization of the Report	8
2 LITERATURE SURVEY	10
2.1 Existing Interview Platforms	10
2.2 AI-Based Recruitment Systems	11
2.3 Research Gap	12
2.4 Proposed Solution	14
3 REQUIREMENTS	16
3.1 Software Requirements	16
3.2 Hardware Requirements	17
3.3 Functional Requirements	19
3.4 Non-Functional Requirements	21
4 SYSTEM DESIGN & METHODOLOGY	24
4.1 System Architecture	24
4.2 Development Methodology	26
4.3 Workflow	27
4.4 Flowchart	29

5	IMPLEMENTATION	32
5.1	User Authentication	32
5.2	AI Mock Interview Module	33
5.3	Competitive Coding Battle	34
5.4	Peer Mock Interview	36
5.5	Recruiter Live Interview Portal	37
5.6	Career Roadmap & AI Coach	38
5.7	Database Design	39
5.8	API Integration	41
6	RESULTS AND ANALYSIS	44
6.1	Functional Testing	44
6.2	Performance Analysis	46
6.3	AI Interview Evaluation	48
6.4	Recruiter Module Results	49
6.5	User Feedback	50
6.6	Discussion	52
7	CONCLUSION AND FUTURE SCOPE	54
7.1	Conclusion	54
7.2	Future Scope	55
	REFERENCES	58
	APPENDICES	59
7.2.1	Appendix A - Source Code Snippet Summary	59
7.2.2	Appendix B - Screenshot Description	59
7.2.3	Appendix C - API Endpoints	60
7.2.4	Appendix D - User Manual	61

CHAPTER - 1

INTRODUCTION

Chapter 1: Introduction

1.1 Introduction to Interview Simulation

Interview simulation is a structured practice process where a candidate experiences the pressure and sequence of a real interview before attending an actual recruitment round. In a technical interview, the candidate must explain concepts, solve problems, communicate thought process, maintain camera presence, and respond to follow-up questions. A useful simulation must therefore measure more than whether an answer is right or wrong.

This project creates a browser-based simulation environment for software interview preparation. The system combines AI-generated questions, spoken answers, camera-based face monitoring, voice analytics, coding practice, peer sessions, recruiter rooms, roadmaps, and final reports. These parts are connected through one dashboard so the candidate can practice repeatedly and track improvement.

1.2 Artificial Intelligence in Recruitment

Artificial intelligence is used in the project to support language-heavy tasks such as generating questions, evaluating answers, preparing coaching responses, and producing personalized learning roadmaps. The AI layer does not replace the complete interview process; instead, it assists the platform by giving consistent practice material and structured feedback.

The backend controls all AI calls so that API keys stay secure. This also gives the server one place to manage prompts, fallback responses, answer evaluation, and stored report data. The result is a safer and more maintainable design than directly calling AI services from the browser.

1.3 Problem Statement

Students often prepare interviews using disconnected tools. They use coding platforms for algorithms, video apps for mock calls, documents for notes, and websites for learning. Because these tools are separate, the student does not receive a single report that connects technical accuracy, communication quality, confidence, camera presence, and learning gaps.

Recruiters also need better organization during online technical assessment. Scheduling, candidate video, coding editor, resume viewing, scoring, and report generation are often handled manually. This increases time and reduces consistency. The project addresses this by providing one platform for practice and evaluation.

1.4 Objectives of the Project

- Develop a secure full-stack platform for interview preparation and recruiter evaluation.
- Generate role-based and topic-based AI interview questions.
- Capture voice answers and calculate communication metrics.
- Use browser camera analysis to detect candidate visibility.
- Provide coding interview and competitive battle practice.
- Enable peer mock interviews and recruiter live interview rooms.
- Generate reports, roadmaps, and AI coach guidance for improvement.

1.5 Scope of the Project

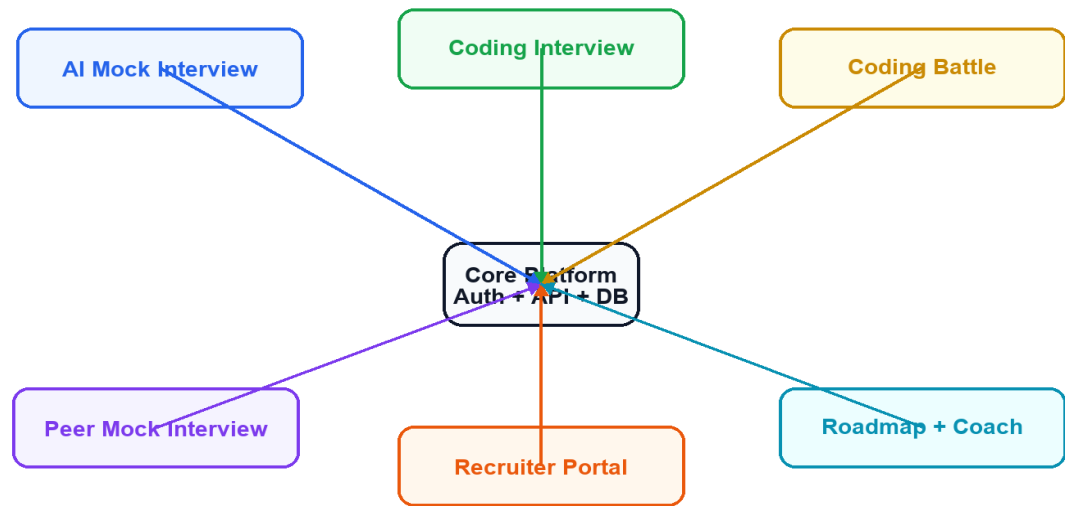
The scope includes software interview preparation, candidate performance tracking, coding practice, peer collaboration, recruiter room workflow, resume support, and learning guidance. The project is built for

academic and placement-preparation use. It does not claim to be a final hiring decision engine; instead, it provides structured practice and supporting evaluation data.

1.6 Organization of the Report

The report is organized into introduction, literature survey, requirements, system design, implementation, results, conclusion, references, and appendices. Each chapter focuses on a different view of the same project: why it is needed, how it is designed, how it is implemented, and how it performs.

Module Interaction Map



Overall module interaction map of the project

CHAPTER - 2

LITERATURE SURVEY

Chapter 2: Literature Survey

2.1 Existing Interview Platforms

Most existing platforms focus on a single activity. Coding platforms help users practice algorithms but usually do not evaluate spoken explanation or camera behavior. Video meeting tools support live conversation but do not store structured interview metrics. Resume tools help manage candidate documents but do not simulate the pressure of a technical interview.

Learning websites provide tutorials and exercises, but they are not designed to behave like an interviewer. A student may know the topic after reading, but still struggle to answer clearly in a timed interview. This difference between learning and interview performance is the main reason a simulation platform is useful.

2.2 AI-Based Recruitment Systems

AI recruitment systems are commonly used for resume screening, chatbot communication, and automated assessment. These systems can reduce manual work, but many are designed for enterprise screening rather than student practice. A preparation-focused system should be transparent, repeatable, and useful even when a candidate is practicing alone.

2.3 Research Gap

The gap observed is that interview preparation tools rarely combine AI questioning, spoken answers, face monitoring, coding practice, peer collaboration, recruiter evaluation, and learning roadmaps in one system. Because these features are usually isolated, candidates cannot easily connect their weaknesses with a continuous improvement plan.

2.4 Proposed Solution

The proposed system fills this gap with a unified web application. It allows users to create accounts, verify email, attend AI interviews, solve coding problems, join coding battles, invite peers, attend recruiter rooms, upload resumes, generate roadmaps, and chat with an AI coach. All modules are connected to persistent user data and reports.

Current Method	Problem Observed	Project Response
Only coding websites	No communication or camera feedback	Coding workspace with face status and interview context
Only video meetings	No automated scoring or report	Interview room stores responses and generates reports
Manual peer practice	Hard to organize sessions	Peer invitation and room workflow
Recruiter notes	Inconsistent evaluation	Recruiter scoring and report actions
Static study plans	Weak topics not connected to practice	AI roadmap and coach support

CHAPTER - 3

REQUIREMENTS

Chapter 3: Requirements

3.1 Software Requirements

Area	Technology	Reason
Frontend	React, Vite, Tailwind CSS	Build responsive pages and production UI
Backend	Node.js, Express.js	Expose REST APIs and control workflow
Database	MongoDB, Mongoose	Store nested interview, report, and roadmap data
AI	Gemini API	Generate questions, evaluate answers, coach users
Realtime	Socket.IO, WebRTC	Support battles, peer rooms, recruiter rooms
Face and Voice	face-api.js, Web Speech API, SpeechSynthesisUtterance	Camera monitoring, speech-to-text, interviewer voice
Security	JWT, bcryptjs, helmet, cors	Protect user accounts and API access
Email and Upload	nodemailer, multer, pdf-parse, mammoth	OTP emails and resume parsing

3.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core CPU	Quad-core or above
Memory	4 GB RAM	8 GB or above
Camera	Basic webcam	HD webcam
Microphone	Built-in microphone	Noise-reduced microphone
Network	Stable internet	Broadband connection
Browser	Chrome or Edge	Latest Chrome/Edge for speech support

3.3 Functional Requirements

- Account registration, email verification, login, and password recovery.
- Interview setup by role, topic, difficulty, and question count.
- AI question delivery with spoken output and answer capture.
- Voice transcript storage and voice metric calculation.
- Camera activation and face status monitoring.
- Coding problem workspace with examples and test cases.
- Peer interview invitations and recruiter live room workflow.
- Roadmap generation, AI coach, dashboard, and report viewing.

3.4 Non-Functional Requirements

The platform should be secure, usable, responsive, maintainable, and deployable. Security is handled using password hashing, JWT tokens, CORS configuration, and server-side AI keys. Usability is supported through clear screens, visible camera/microphone states, and direct navigation. Maintainability is improved by separating pages, hooks, routes, controllers, and models by feature.

CHAPTER - 4

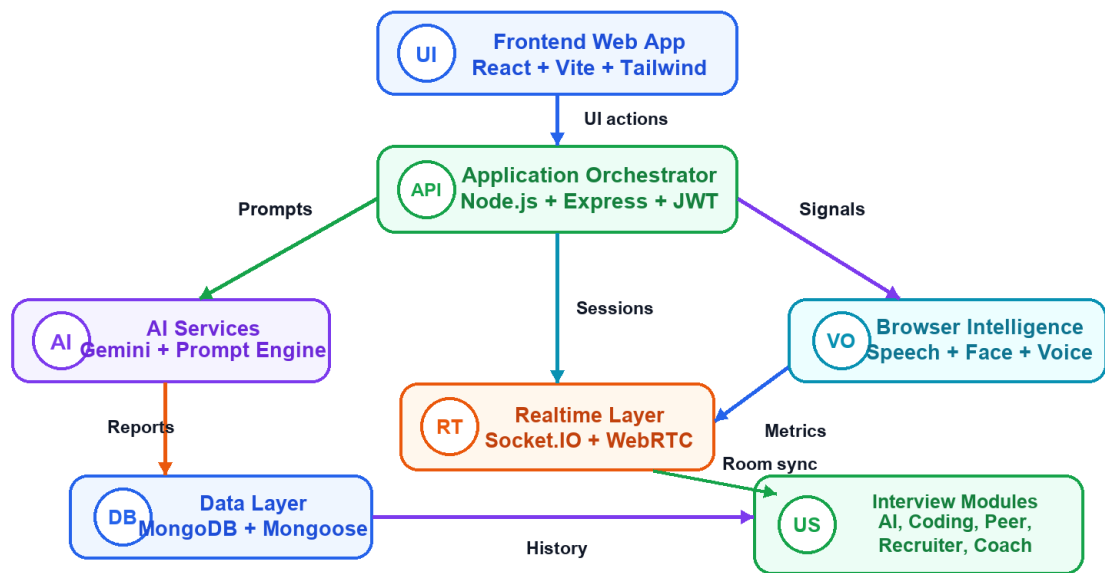
SYSTEM DESIGN AND METHODOLOGY

Chapter 4: System Design and Methodology

4.1 System Architecture

The architecture contains six major layers: frontend web app, backend application orchestrator, AI services, browser intelligence, data layer, and real-time layer. The frontend manages UI and browser APIs. The backend validates requests and coordinates storage and AI services. MongoDB stores persistent data. Socket.IO and WebRTC support live interactions.

Architecture Diagram



High-level project architecture

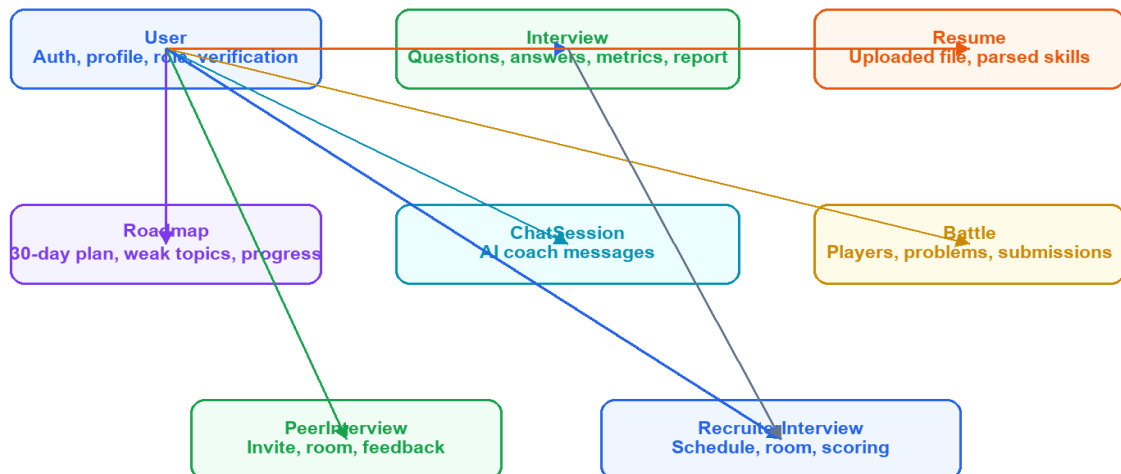
4.2 Development Methodology

The implementation followed an incremental approach. Authentication was built first because all protected modules depend on user identity. After that, interview setup and live interview flow were developed. Face analysis, speech recognition, voice metrics, coding workspace, battle rooms, peer rooms, recruiter portal, roadmap, coach, and deployment support were added step by step.

4.3 Workflow

In the AI interview workflow, the user logs in, selects interview settings, grants camera and microphone permission, listens to the AI question, answers by voice or typing, submits the response, and receives the next question. After completion, the report summarizes content and communication performance.

Database Schema Overview

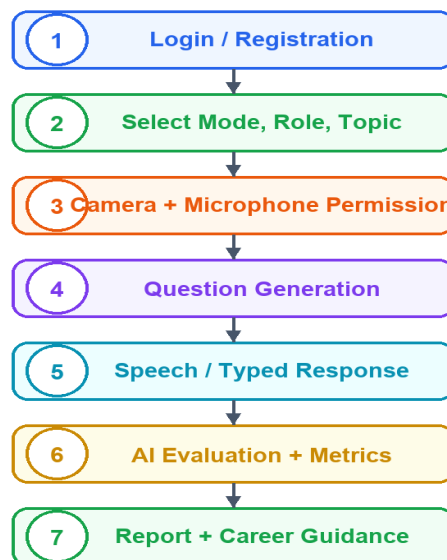


AI interview workflow

4.4 Database Schema

The database design uses separate collections for users, interviews, resumes, roadmaps, chat sessions, battles, peer invitations, peer interviews, recruiter interviews, and schedules. This division makes the project easier to understand because each feature has its own data responsibility.

AI Interview Workflow



Database schema overview

CHAPTER - 5

IMPLEMENTATION

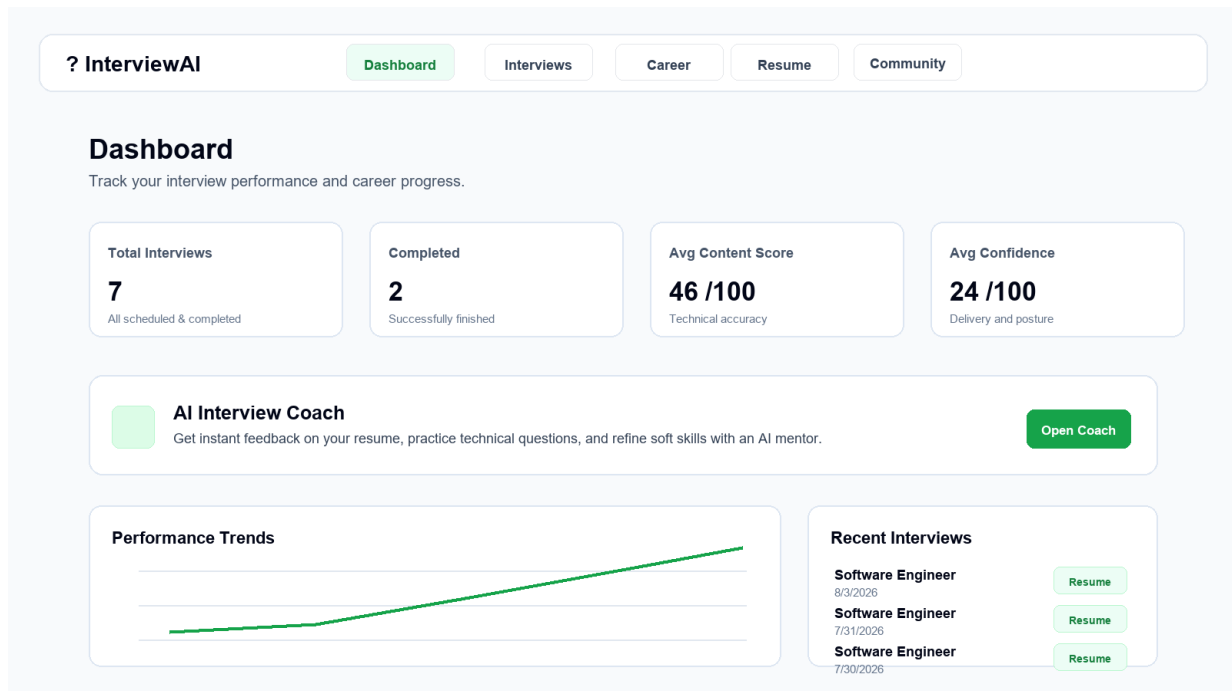
Chapter 5: Implementation

5.1 User Authentication

The authentication module includes register, login, Google login endpoint, OTP verification, resend OTP, forgot password, reset password, and current user profile. Passwords are protected with bcryptjs. JWT tokens allow the frontend and backend to identify authenticated users for protected pages and APIs.

5.2 Dashboard Module

The dashboard displays interview count, completed sessions, average content score, confidence score, recent interviews, performance trends, and AI coach access. It works as the central entry page after login and helps the user understand current preparation progress.



Dashboard screenshot with performance summary and recent interviews

5.3 AI Mock Interview Module

The AI interview module asks questions based on role, topic, difficulty, and interview type. The interviewer avatar speaks the question, the microphone starts after speech ends, and the candidate answer is submitted with transcript, face metrics, and voice metrics. The backend stores the answer and prepares the next question without repeating completed questions.

5.4 Face Recognition and Proctoring

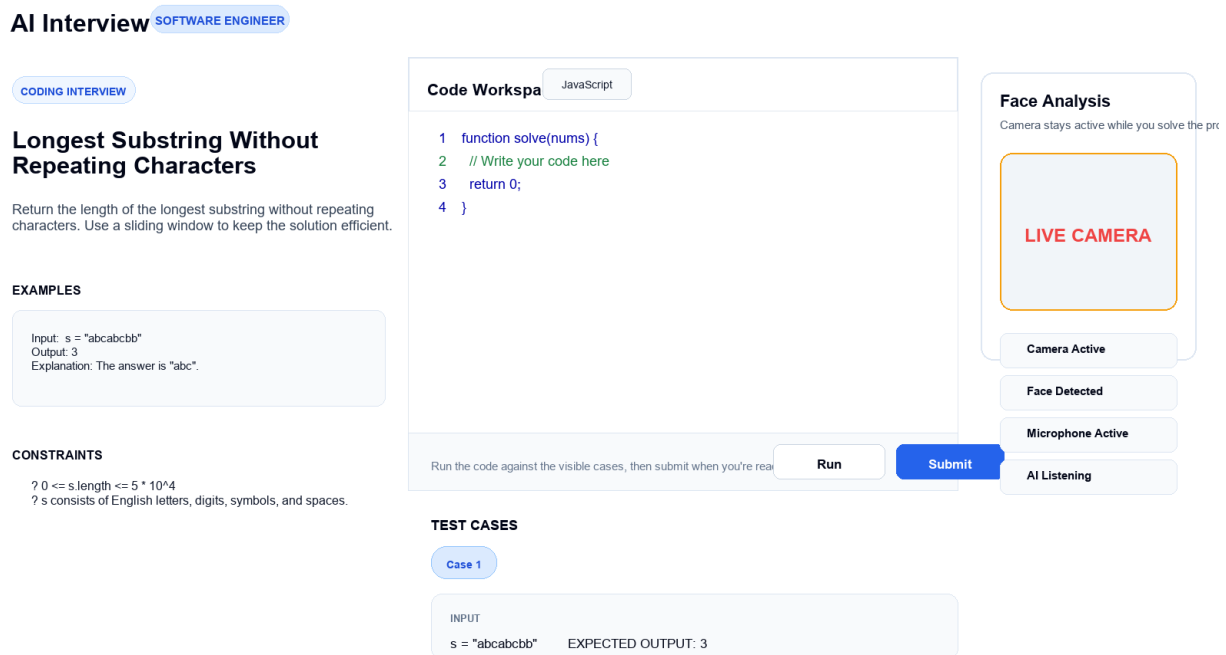
Face monitoring uses face-api.js in the browser. TinyFaceDetector checks whether a face is visible. FaceLandmark68Net supports face positioning checks, and FaceExpressionNet can support expression-related signals. The system shows states such as Face Detected, No Face Detected, Multiple Faces Detected, and Camera Error. Grace periods reduce false warnings from brief movement.

5.5 Voice Recognition and Voice Analytics

Voice recognition uses the Web Speech API with Indian English configuration. continuous recognition and interim results are enabled so longer answers can be captured more smoothly. SpeechSynthesisUtterance speaks the interviewer questions and prefers en-IN voices when available. The voice analysis utility calculates WPM, filler count, pause count, clarity, fluency, and confidence.

5.6 Coding Interview Module

The coding interview page provides a problem statement, examples, constraints, code workspace, language selector, separate Run and Submit buttons, test cases, and face analysis panel. This design supports LeetCode-style practice while keeping camera monitoring active during the interview.



Coding interview screenshot with test cases and face analysis

5.7 Competitive Coding Battle

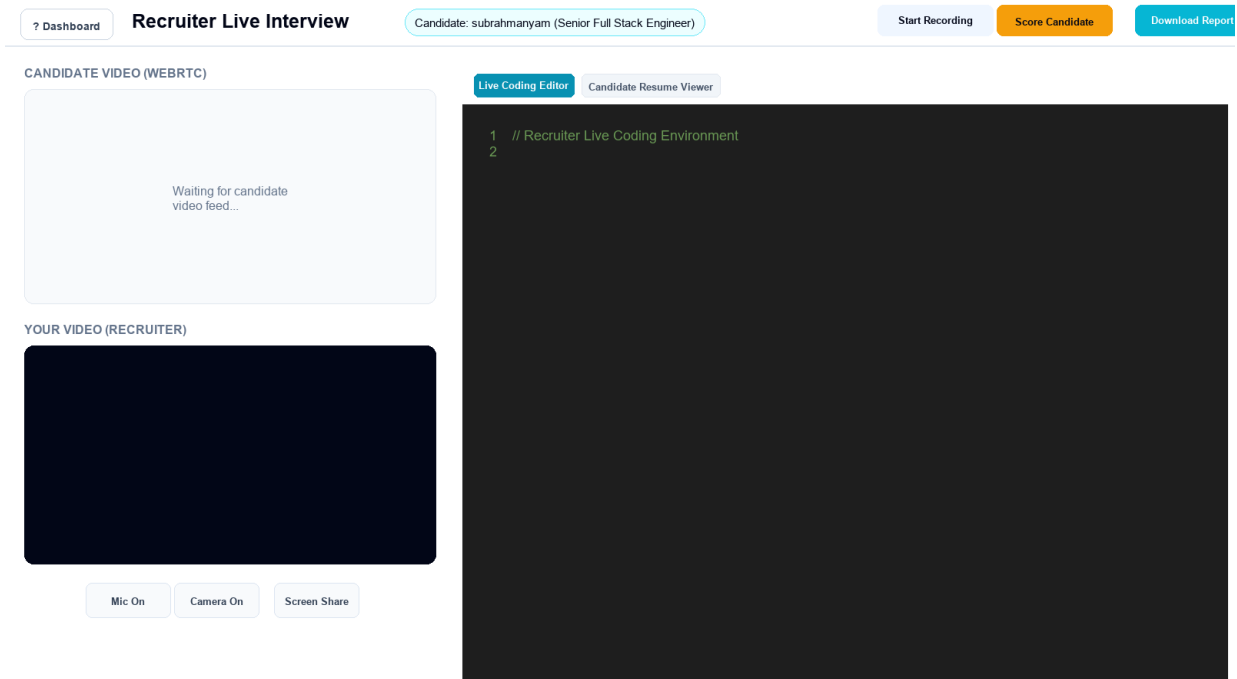
The coding battle module adds competitive practice. Users can enter battle rooms, solve coding problems, and submit answers under pressure. Socket.IO is used to keep battle state synchronized between participants.

5.8 Peer Mock Interview

The peer interview module allows users to search candidates, update profile information, send invitations, respond to requests, open interview rooms, submit feedback, and view history. It is useful because students can improve by practicing both interviewer and interviewee roles.

5.9 Recruiter Live Interview Portal

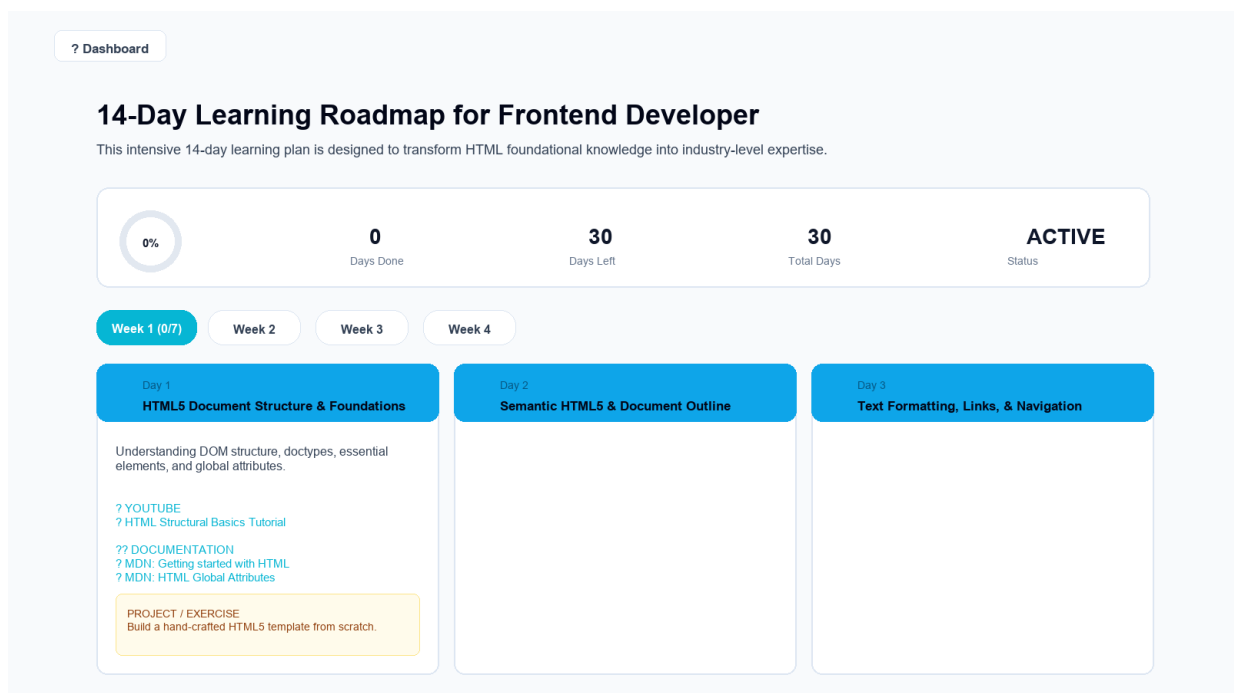
The recruiter portal supports scheduling, room access, candidate video area, recruiter video area, live coding editor, resume viewer, score candidate action, recording status, and report download. This module connects the preparation platform with professional assessment requirements.



Recruiter portal screenshot with live coding editor and interview controls

5.10 Career Roadmap and AI Coach

The roadmap module converts weak topics into a learning plan with days, resources, projects, and progress tracking. The AI coach provides chat-based interview guidance and technical explanation. Together, these features help the candidate improve after receiving interview feedback.



Roadmap screenshot with day-wise learning cards

5.11 API Integration

API Group	Main Endpoints	Use
Auth	/register, /login, /verify-otp, /reset-password	Account and verification
Interviews	/setup, /:id/answer, /:id/report	Interview lifecycle

API Group	Main Endpoints	Use
Code	/run, /submit	Coding evaluation
Roadmaps	/generate, /:id/day/:day/complete	Learning plan
Coach	/sessions, /message	AI mentor chat
Peer	/candidates, /invite, /room/:roomId	Peer practice
Recruiter	/schedule, /interviews, /room/:roomId/score	Recruiter workflow
Resume	/upload, /my-resumes	Resume handling

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

5.5 Recruiter Live Interview Portal

Recruiter Live Interview Portal is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of professional hiring workflow, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are schedule creation, recruiter dashboard, live room, scoring, recording metadata. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well

as a foundation for future production improvements.

From a software engineering point of view, recruiter live interview portal is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

5.6 Career Roadmap & AI Coach

Career Roadmap & AI Coach is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of continuous preparation support, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI

question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are roadmap generation, roadmap progress, coach sessions, chat messages. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, career roadmap & ai coach is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

5.7 Database Design

Database Design is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In

this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of persistent storage, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are User, Interview, Resume, Roadmap, ChatSession, Battle, PeerInterview, RecruiterInterview models. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, database design is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible

screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

Collection	Main Data Stored	Used By
users	Name, email, password hash, roles, verification state	Authentication and authorization
interviews	Setup, questions, answers, metrics, report	AI interview and reports
resumes	Uploaded document metadata and parsed text	Resume-based personalization
roadmaps	Generated learning plan, weak topics, progress	Career roadmap
chatSessions	Coach conversation and messages	AI coach
battles	Players, problems, submissions, result	Coding battle
peerInterviews	Peer room, participants, feedback	Peer mock interview
recruiterInterviews	Schedule, recruiter room, candidate score	Recruiter portal

5.8 API Integration

API Integration is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of frontend-backend connectivity, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are auth routes, interview routes, code routes, peer routes, recruiter routes, roadmap routes. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, api integration is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

API Group	Base Route	Important Endpoints
Authentication	/api/auth	register, login, google, verify-otp, resend-otp, forgot-password, reset-password, me
Interviews	/api/interviews	setup, history/me, answer, skip, complete, report, proctoring-violation
Code Runner	/api/code	run, submit
Roadmaps	/api/roadmaps	generate, list, detail, complete day, delete
Coach	/api/coach	sessions, messages, update, delete
Peer	/api/peer	candidates, invite, invitations, room, feedback, history
Recruiter	/api/recruiter	schedule, interviews, room, score, recording
Resume	/api/resumes	upload, parse, list, delete
Admin	/api/admin	admin dashboard and management operations

CHAPTER - 6

RESULTS AND ANALYSIS

6.1 Functional Testing

Functional Testing is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of feature verification, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are auth, AI interview, coding, battle, peer, recruiter, roadmap, coach. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, functional testing is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

Test ID	Module	Scenario	Expected Result	Status
TC-01	Authentication	Register with valid email and password	OTP sent and verification screen shown	Pass
TC-02	Authentication	Login before verification	Meaningful verification error shown	Pass
TC-03	AI Interview	Start role-based interview	First question displayed and answer accepted	Pass
TC-04	Coding Interview	Run sample solution	Visible test result section updates	Pass
TC-05	Coding Battle	Join battle lobby	Battle room opens with problem and editor	Pass
TC-06	Peer Interview	Send peer request	Invitation appears for receiver	Pass

Test ID	Module	Scenario	Expected Result	Status
TC-07	Recruiter Portal	Schedule interview	Room and candidate details are stored	Pass
TC-08	Roadmap	Generate roadmap by weak topic	Roadmap card and daily plan are created	Pass

6.2 Performance Analysis

Performance Analysis is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of runtime observations, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are page load, API calls, database persistence, real-time communication. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, performance analysis is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

Module	Observed Operation	Performance Consideration	Optimization Used
Frontend	Route navigation and dashboards	Should remain responsive	Vite build and component-based pages
API Server	Authenticated REST requests	Should respond predictably	Express middleware and route separation
Database	Interview and session persistence	Should support nested results	Mongoose schema design
AI Service	Question and roadmap generation	Dependent on quota and response time	Backend-only calls and fallback handling
Realtime	Peer and battle rooms	Requires low latency	Socket.IO and WebRTC support

6.3 AI Interview Evaluation

AI Interview Evaluation is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of quality of AI-supported feedback, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are answer content, communication, confidence, report narrative. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, ai interview evaluation is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

Metric	Meaning	How It Helps Candidate
Content Score	Measures correctness and completeness of answer	Improves technical preparation
Confidence	Indicates delivery quality and posture signals	Improves communication style
Fluency	Captures response smoothness and hesitation	Reduces interview anxiety
Eye Contact	Estimates focus using browser camera analysis	Encourages professional presence
Recommendation	Summarizes next steps	Provides actionable preparation plan

6.4 Recruiter Module Results

Recruiter Module Results is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of live recruiter workflow, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery,

answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are scheduling, room link, scoring, candidate record. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, recruiter module results is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

6.5 User Feedback

User Feedback is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where can-

didates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of candidate usability observations, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are clear UI, fast navigation, realistic practice, useful reports. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, user feedback is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

6.6 Discussion

Discussion is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of strengths and limitations, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are AI dependency, quota limits, browser permissions, network reliability. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, discussion is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

CHAPTER - 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Conclusion is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of project outcome, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are unified interview simulation, recruitment support, coding practice, reports, AI guidance. Their interaction is designed to reduce manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, conclusion is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persis-

tence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

7.2 Future Scope

Future Scope is a central part of the proposed AI-driven interview simulation system because it connects the user-facing experience with measurable recruitment outcomes. In this project, the feature is implemented as part of a browser-based platform where candidates can prepare, recruiters can evaluate, and peers can collaborate without switching between separate tools.

The implementation uses React, Vite, and Tailwind CSS on the client side to keep the user interface responsive and consistent. The backend uses Node.js and Express to expose secure REST APIs, while MongoDB stores users, interview sessions, responses, reports, roadmaps, rooms, and activity history. This separation keeps presentation logic, business logic, and data persistence understandable for maintenance and review.

In the context of possible enhancements, the system captures structured inputs from the user and converts them into useful interview actions. These actions include authentication, interview configuration, camera and microphone activation, AI question delivery, answer submission, code execution, peer connection, recruiter scoring, and report generation depending on the selected module.

The major modules involved are multi-provider AI, stronger code judge, analytics dashboard, mobile app, institution admin tools. Their interaction is designed to reduce

manual effort while preserving the natural flow of an interview. AI services assist with question generation and coaching, but the application still keeps final data storage, permissions, and workflow control inside the backend server.

The design also considers practical constraints such as browser permissions, network reliability, free deployment limitations, and API quota limits. Therefore, fallback behavior, clear error messages, and modular routing are important engineering decisions in the project. These decisions make the platform suitable for academic demonstration as well as a foundation for future production improvements.

From a software engineering point of view, future scope is not treated as an isolated screen. It is connected to routing, protected access, backend validation, database persistence, and user feedback. This approach prevents a common problem in student projects where screens look complete but do not share consistent state or saved records.

The project follows a practical full-stack pattern in which the client sends only the required data to the server and the server performs sensitive operations such as authentication, AI API calls, and database writes. This keeps credentials and API keys away from the browser and provides one controlled place to apply error handling, logging, and validation.

For review and testing, this part of the system can be demonstrated through a clear sequence of actions: open the module, submit valid input, observe the server response, inspect stored history, and verify the final user-facing output. This sequence makes the implementation easier to explain during viva and final evaluation because each visible screen maps to a backend responsibility.

The design also improves future maintainability. Since routes, models, and pages are organized by feature, new capabilities such as advanced analytics, stronger coding judge support, multi-provider AI, institution dashboards, or recruiter comparison reports can be added without rewriting the whole application.

- Add support for multiple AI providers to reduce dependency on a single quota-limited service.
- Improve code execution with sandboxed language containers and hidden test-case scoring.
- Add institution dashboards for faculty to monitor student readiness.
- Add advanced recruiter analytics and candidate comparison reports.

- Provide mobile-friendly or native app support for practice reminders and roadmap tracking.

REFERENCES

- [1] React Documentation, Component-based user interface development.
- [2] Vite Documentation, Frontend build tooling and development server.
- [3] Tailwind CSS Documentation, Utility-first responsive styling.
- [4] Node.js Documentation, JavaScript runtime for backend development.
- [5] Express.js Documentation, REST API routing and middleware.
- [6] MongoDB and Mongoose Documentation, NoSQL database design and schema modelling.
- [7] Socket.IO Documentation, Real-time bidirectional event communication.
- [8] WebRTC Documentation, Browser-based audio/video peer communication.
- [9] Web Speech API Documentation, Speech recognition and synthesis in browser environments.
- [10] Google Gemini API Documentation, Generative AI model integration.
- [11] JWT and bcrypt documentation, authentication and password security.
- [12] Render Documentation, full-stack Node.js web service deployment.

APPENDICES

7.2.1 Appendix A - Source Code Snippet Summary

The following appendix lists the major files used in the implementation without reproducing the full source code. This keeps the report concise while still showing the relationship between folders and features.

Area	Important Files	Purpose
Client routing	client/src/App.jsx	Defines application pages and protected routes
Authentication UI	LoginPage.jsx, RegisterPage.jsx, VerifyEmailPage.jsx	Login, registration, and OTP verification interfaces
Dashboard	DashboardPage.jsx	Performance cards, coach entry, trends, recent interviews
Live interviews	InterviewSetupPage.jsx, LiveInterviewPage.jsx, ReportPage.jsx	Setup, question flow, answering, final report
Coding modules	BattleLobbyPage.jsx, BattleArenaPage.jsx	Coding battle lobby and arena
Peer modules	PeerCandidatesPage.jsx, PeerInterviewRoom- Page.jsx	Peer discovery, room, history
Recruiter modules	RecruiterDashboardPage.jsx, RecruiterRoomPage.jsx	Recruiter schedule and live room
Backend routing	server/src/routes/*.js	REST endpoints for all modules
Backend models	server/src/models/*.js	MongoDB schemas for persistence

7.2.2 Appendix B - Screenshot Description

AI Interview Dashboard: Displays performance summary cards, recent interviews, trend chart, and AI coach entry.

Coding Interview Interface: Displays problem description, code editor, run/submit

actions, test cases, and face analysis.

Coding Battle Room: Displays competitive programming problem and synchronized battle state.

Peer Interview Module: Allows candidates to discover users, send requests, and enter peer mock rooms.

Recruiter Dashboard: Allows recruiter scheduling, interview room entry, and candidate scoring.

AI Career Coach: Provides conversational interview guidance and technical preparation help.

7.2.3 Appendix C - API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Create account and begin verification workflow
POST	/api/auth/login	Authenticate verified user
POST	/api/auth/google	Google OAuth based authentication
POST	/api/auth/verify-otp	Verify registration OTP
POST	/api/auth/resend-otp	Send OTP again
POST	/api/auth/forgot-password	Send password reset email
POST	/api/auth/reset-password	Set new password
GET	/api/auth/me	Return logged-in user profile
POST	/api/interviews/setup	Create AI interview session
POST	/api/interviews/:id/answer	Submit answer for a question
POST	/api/interviews/:id/complete	Complete interview and prepare report
GET	/api/interviews/:id/report	Read interview report
POST	/api/code/run	Run code against examples
POST	/api/code/submit	Submit coding solution
POST	/api/roadmaps/generate	Generate personalized roadmap
POST	/api/peer/invite	Send peer interview invitation

Method	Endpoint	Purpose
GET	/api/recruiter/interviews	List recruiter interviews
POST	/api/recruiter/schedule	Schedule recruiter interview

7.2.4 Appendix D - User Manual

- Open the application and create an account using name, email, and password.
- Verify the account by entering the OTP received through email.
- Login and open the dashboard to view performance history and available modules.
- Choose AI Interview or Coding Interview from the Interviews menu.
- Configure role, topic, difficulty, and question count before starting.
- Allow camera and microphone permissions for live interview monitoring.
- Answer questions through speech or typing, then submit responses.
- Complete the interview and view the generated report.
- Use Coding Battle or Peer Mock Interview from the Community menu for collaborative practice.
- Use Career Roadmap and AI Coach from the Career menu for long-term preparation.